Report:

Minimum Loss

1. Introduction

This report explains the working and structure of the provided Java program named 'Solution.java'. The program is designed to read a sequence of integer values and compute the minimum positive difference between a number and a greater number that has already appeared earlier in the sequence. The program uses efficient input/output techniques and a TreeSet data structure from the Java Collections Framework to maintain sorted values and perform fast lookups.

The code resembles templates often used in competitive programming environments. Such templates include fast input/output handling and reusable helper functions to simplify reading numbers from input. This report explains each part of the code, the purpose of imported packages, the logic used in the algorithm, and a complete time and space complexity analysis.

2. Imported Packages and Their Explanation

java.awt.*: The Abstract Window Toolkit (AWT) package provides classes used to create graphical user interfaces (GUI). It includes components like buttons, labels, windows, layouts, and graphics drawing tools. In this particular program, these classes are not used directly but may appear in general programming templates.

java.awt.event.*: This package contains classes and interfaces for handling events generated by GUI components. For example, it supports mouse clicks, keyboard actions, and button presses. Although it is imported here, it is not used in this specific program.

java.awt.geom.*: This package provides advanced geometric shapes and operations such as lines, rectangles, curves, and transformations. These are useful in graphical and visualization applications but are not required for this algorithm.

java.io.*: This package contains classes used for input and output operations. The program specifically uses BufferedReader, InputStreamReader, BufferedWriter, OutputStreamWriter, and PrintWriter. These classes allow fast reading and writing of data.

java.math.*: This package contains classes for performing high-precision mathematical calculations. It includes classes like BigInteger and BigDecimal which support numbers larger than the primitive numeric types. These are not used directly in this program.

java.text.*: This package provides classes for formatting and parsing text, numbers, and dates. It includes utilities such as DecimalFormat and SimpleDateFormat. It is imported in the template but not used in this code.

java.util.*: This is one of the most important packages used in the program. It provides utility classes such as TreeSet, StringTokenizer, and other collection framework classes used for data structures and algorithms.

3. Important Classes and Data Structures Used

BufferedReader: BufferedReader is used to read text from the input stream efficiently. It reads large chunks of data into a buffer and then processes them, which reduces the number of input operations and improves performance.

PrintWriter: PrintWriter is used for output. It allows formatted printing of data to the console or a file. When combined with BufferedWriter, it improves the efficiency of output operations.

StringTokenizer: This class is used to split input strings into smaller tokens. For example, if a line contains multiple numbers separated by spaces, StringTokenizer allows reading them one by one.

TreeSet: TreeSet is a collection that stores elements in sorted order. It is implemented using a balanced binary search tree (Red-Black Tree). This allows operations like insertion, deletion, and searching to be performed in logarithmic time.

4. Structure of the Program

The program contains the following main components:
1. Import statements
2. Class declaration (Solution)
3. Input and output variables
4. The main() method
5. Helper methods for reading input values
6. Logic for computing the minimum difference

5. Algorithm

Step 1: The program starts execution from the main() method.

Step 2: BufferedReader and PrintWriter objects are created to handle fast input and output.

Step 3: The variable qq represents the number of test cases. In this template it is set to Integer.MAX_VALUE so the program keeps reading input until it ends.

Step 4: For each iteration, the program reads an integer n which represents how many numbers will follow.

Step 5: A TreeSet named 'set' is created to store numbers in sorted order.

Step 6: A variable 'ret' is initialized to Long.MAX_VALUE. This variable will store the minimum difference found.

Step 7: A loop runs n times to read each number.

Step 8: For each number 'curr', the program checks if the set already contains elements.

Step 9: If the largest element in the set is greater than the current value, then a candidate difference exists.

Step 10: The function set.higher(curr) finds the smallest element greater than 'curr'.

Step 11: The difference between this element and 'curr' is calculated.

Step 12: The program updates 'ret' using Math.min() if a smaller difference is found.

Step 13: The current number is inserted into the TreeSet.

Step 14: After all numbers are processed, the minimum difference is printed.

5. Code

```
import java.awt.*;

import java.awt.event.*;

import java.awt.geom.*;

import java.io.*;

import java.math.*;

import java.text.*;

import java.util.*;

public class Solution {

        private static BufferedReader br;

        private static StringTokenizer st;

        private static PrintWriter pw;
```

```
public static void main(String[] args) throws IOException    {
                br = new BufferedReader(new InputStreamReader(System.in));
                pw = new PrintWriter(new BufferedWriter(new
OutputStreamWriter(System.out)));
                int qq = Integer.MAX_VALUE;
                for(int casenum = 1; casenum <= qq; casenum++)    {
                        int n = readInt();
                        TreeSet<Long> set = new TreeSet<Long>();
                        long ret = Long.MAX_VALUE;
                        while(n-- > 0) {
                                long curr = readLong();
                                if(!set.isEmpty() && set.last() > curr) {
                                        ret = Math.min(ret, set.higher(curr) - curr);
                                }
                                set.add(curr);
                        }
                        pw.println(ret);
                }
                pw.close();
        }
        private static void exitImmediately()  {
                pw.close();
                System.exit(0);
        }
private static long readLong() throws IOException   {
```

```
        return Long.parseLong(nextToken());
    }
    private static double readDouble() throws IOException    {
        return Double.parseDouble(nextToken());
    }
    private static int readInt() throws IOException    {
        return Integer.parseInt(nextToken());
    }
    private static String nextToken() throws IOException    {
        while(st == null || !st.hasMoreTokens())    {
            if(!br.ready())  {
                exitImmediately();
            }
            st = new StringTokenizer(br.readLine().trim());
        }
        return st.nextToken();
    }
}
```

7. Example Execution

Example Input:
5
10 3 20 8 15

Execution Process:
- Insert 10 → set = {10}
- Insert 3 → next higher = 10 → difference = 7
- Insert 20 → no higher element
- Insert 8 → next higher = 10 → difference = 2
- Insert 15 → next higher = 20 → difference = 5

Minimum difference found = 2

8. Time Complexity Analysis

Let n be the number of elements.

Reading each input value takes constant time O(1).

Checking conditions such as set.isEmpty() and comparisons take O(1).

TreeSet operations are based on a balanced binary search tree. The height of the tree is approximately log n.

set.last() operation takes O(log n) because it navigates through the tree.

set.higher(curr) also takes O(log n) as it searches the tree to find the smallest element greater than the given value.

Inserting an element using set.add(curr) requires O(log n) time to maintain the sorted tree structure.

Since these operations occur inside a loop that runs n times, the overall time complexity becomes:

Total Time Complexity = O(n log n)

9. Space Complexity

The TreeSet stores all input numbers. In the worst case, it may store n elements. Therefore, the space required grows linearly with the number of inputs.

Space Complexity = O(n)

10. Advantages of the Approach

1. Efficient searching due to TreeSet.
2. Automatically maintains sorted order of elements.
3. Logarithmic time operations improve performance for large inputs.
4. Fast input/output operations using BufferedReader and PrintWriter.

11. Conclusion

In conclusion, the provided Java program efficiently determines the minimum positive difference between numbers in a sequence using a TreeSet data structure. The use of balanced tree structures ensures that insertion and search operations remain efficient even for large datasets. With a time complexity of O(n log n) and space complexity of O(n),

the program is well suited for competitive programming and algorithmic problem solving tasks.